Projectile Motion & Coupled ODEs

Songhang

March 14, 2026

```
# Define initial conditions
state_nodrag[0,0]=0                                  # set x at t=0
state_nodrag[1,0]=h                                  # set y at t=0
state_nodrag[2,0]=v0*np.cos(theta*np.pi/180)         # set vx at t=0
state_nodrag[3,0]=v0*np.sin(theta*np.pi/180)         # set vy at t=0
```

Figure 1: the initial conditions and time steps are defined here

```
# Define empty 4xNpts empty arrays for positions and velocities
# > state[0] will be an array of length Npts to store all the x positions
# > state[1] will be an array of length Npts to store all the y positions
# > state[2] will be an array of length Npts to store all the x velocities
# > state[3] will be an array of length Npts to store all the y velocities
state_nodrag = np.zeros((4,Npts))
```

Figure 2: These variables encode the state at each time step

```
# Use a for loop to define the remainder of the positions and velocities
for i in range(1,Npts):
    # calculate the changes to x, y, vx, and vy using the Euler method
    delta_i = df_Euler(state_nodrag[:,i-1],ts[i-1],dt,derivs_NoDrag)

    # calculate the ith value of x, y, vx, and vy by adding the df values
    state_nodrag[:,i] = state_nodrag[:,i-1] + delta_i
```

Figure 3: The derivatives being calculated and updated here

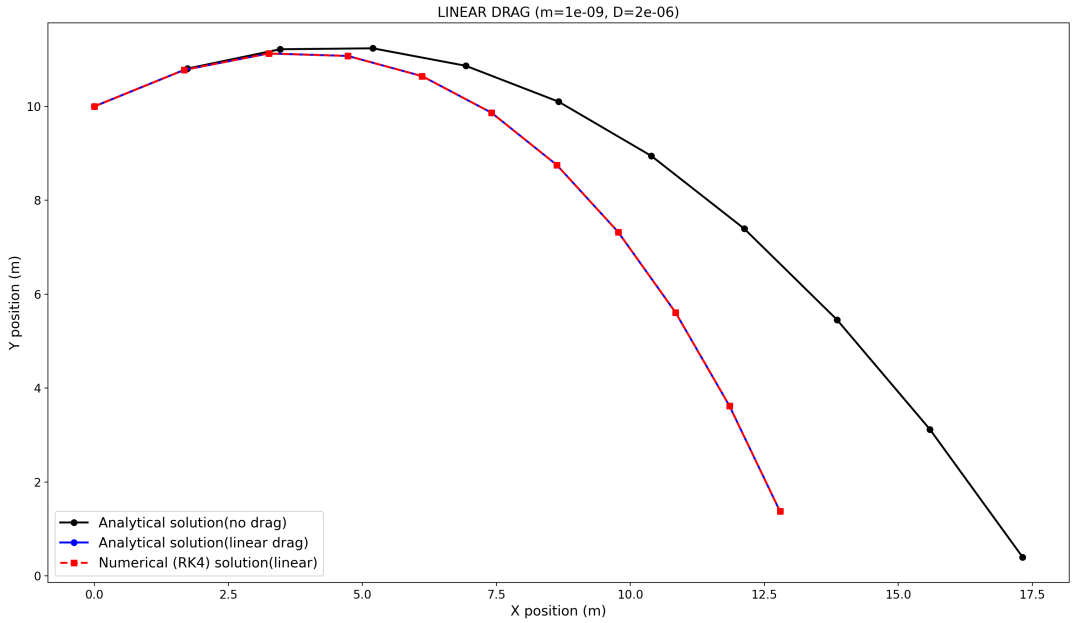

Figure 4: CHECKPOINT: Create a new function derivsLinearDrag that is similar to derivsNoDrag but returns the derivatives $\dot{\mathbf{S}}$ based on Eq. 10.

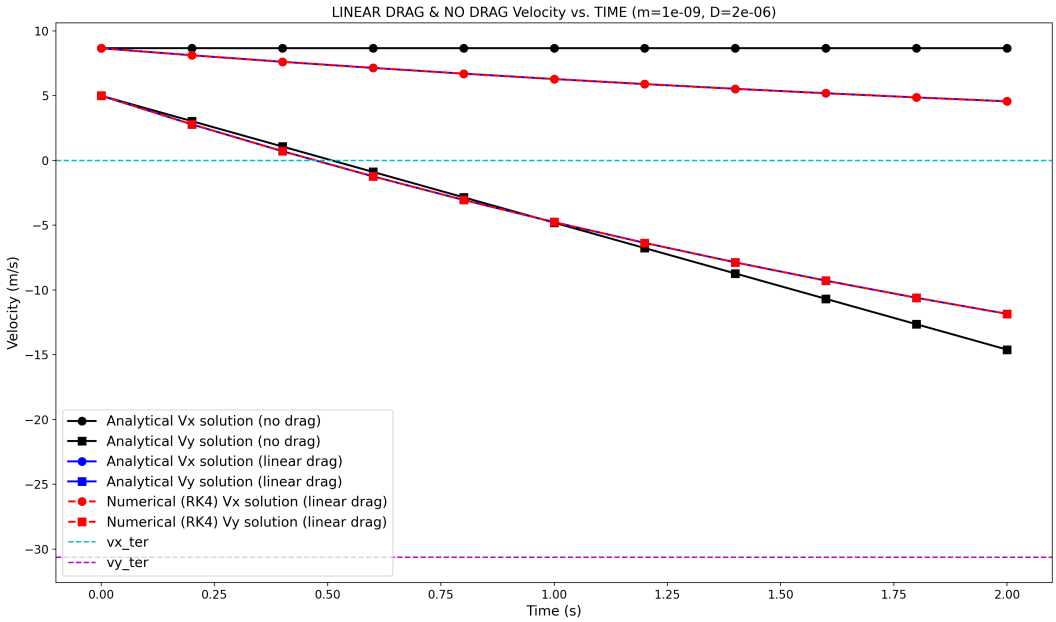

Figure 5: CHECKPOINT: Make a plot of $v_x(t)$ vs. t and $v_y(t)$ vs. t on a shared set of axes.

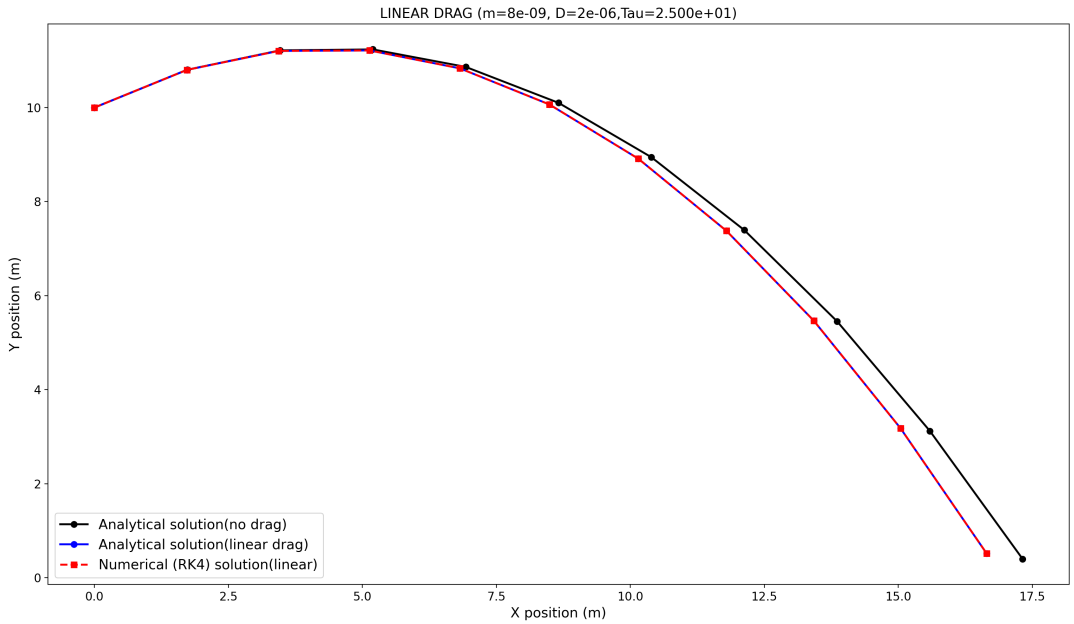

Figure 6: trajectory evolution for the case where the linear drag is negligible

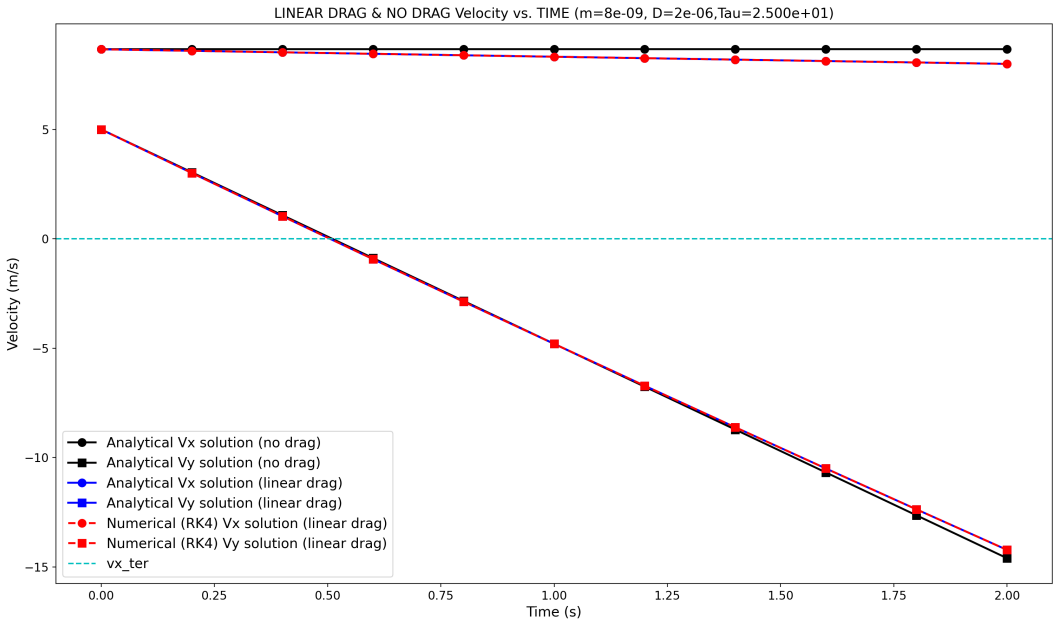

Figure 7: velocity evolution for the case where the linear drag is negligible

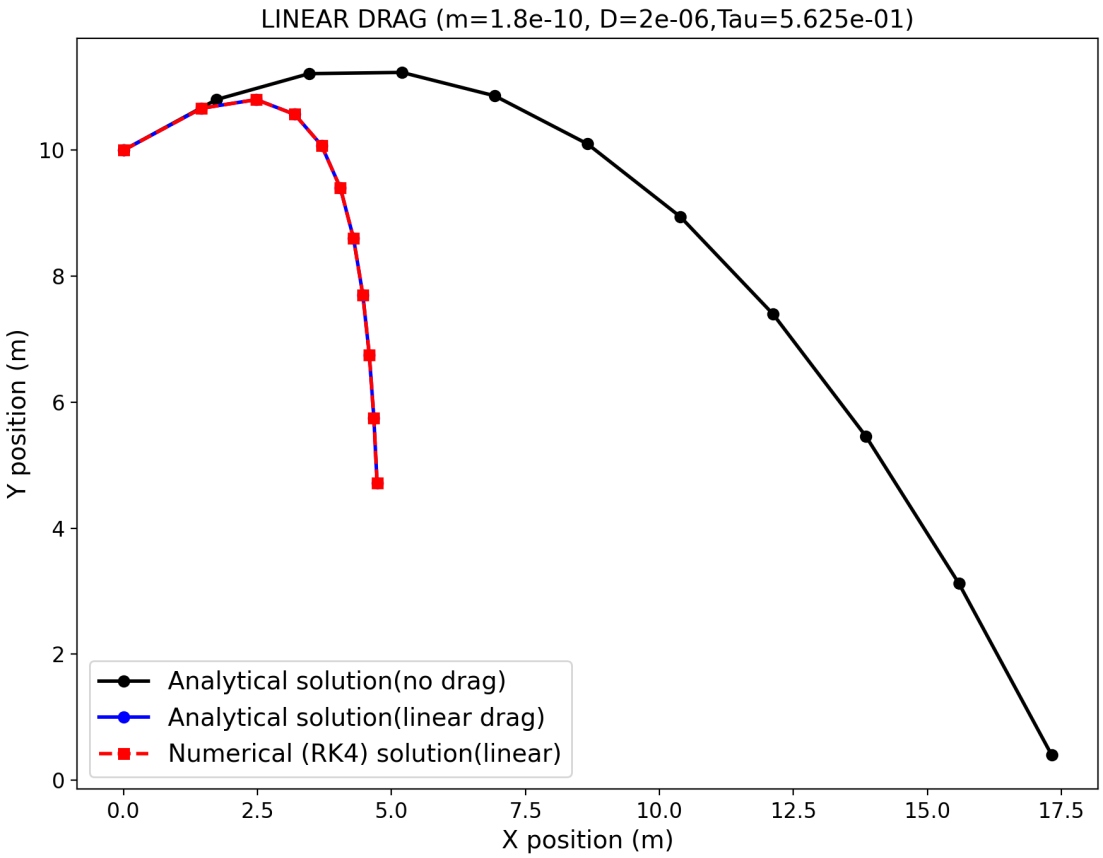

Figure 8: trajectory evolution for the case where the linear drag is not negligible

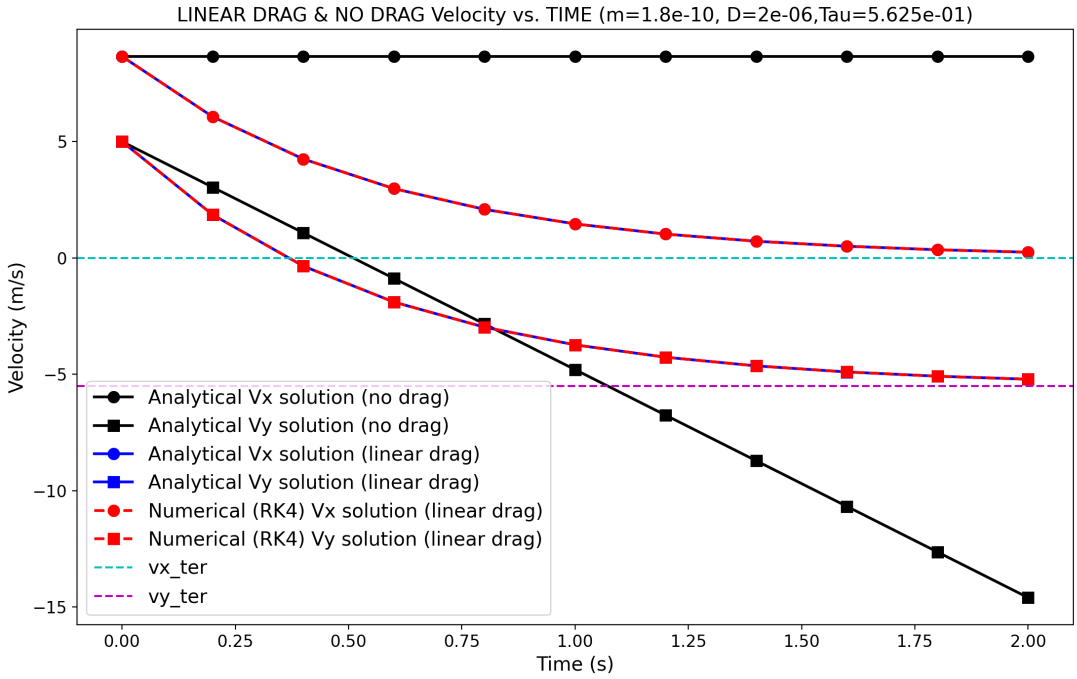

Figure 9: velocities approach the expected limits for $t \to \infty$

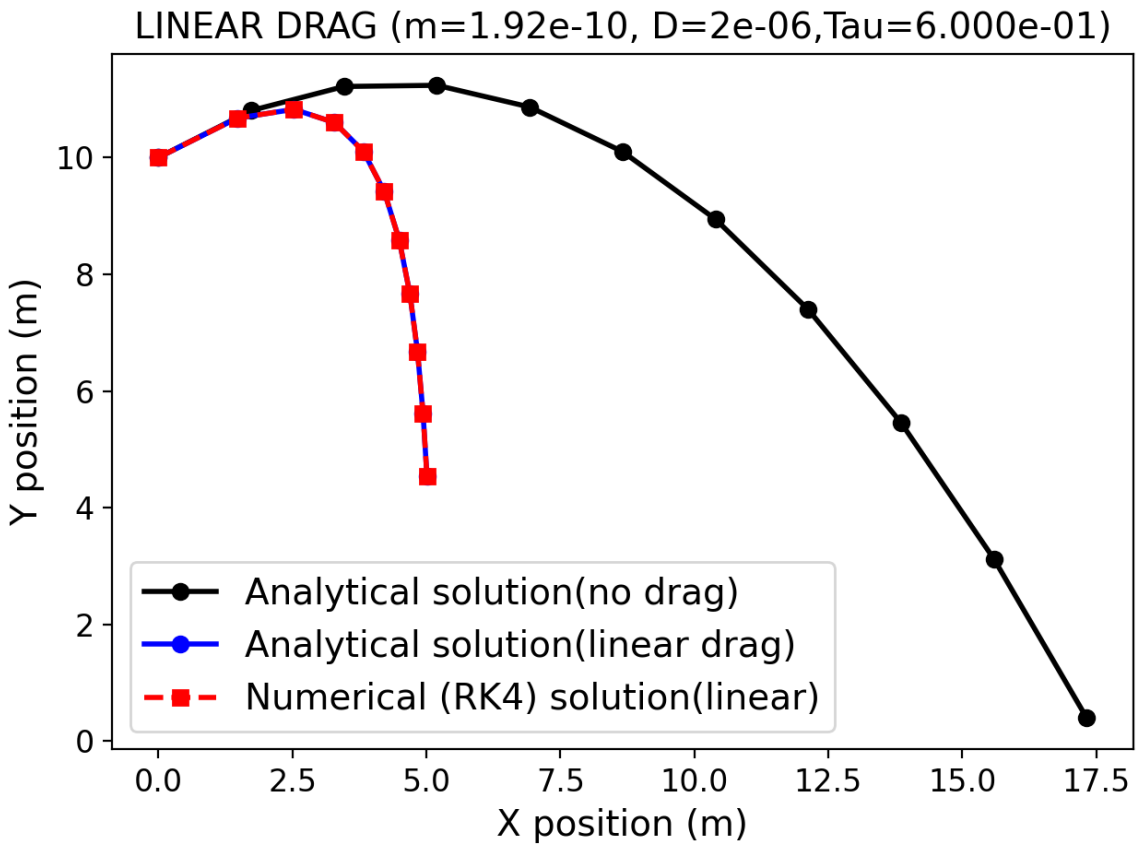

Figure 10: RK4 trajectories with linear drag for $\tau = 3\Delta t\mathrm{m} = 1.92\mathrm{e}-10, \mathrm{D} = 2\mathrm{e}-06,$ Tau $= 0.6$

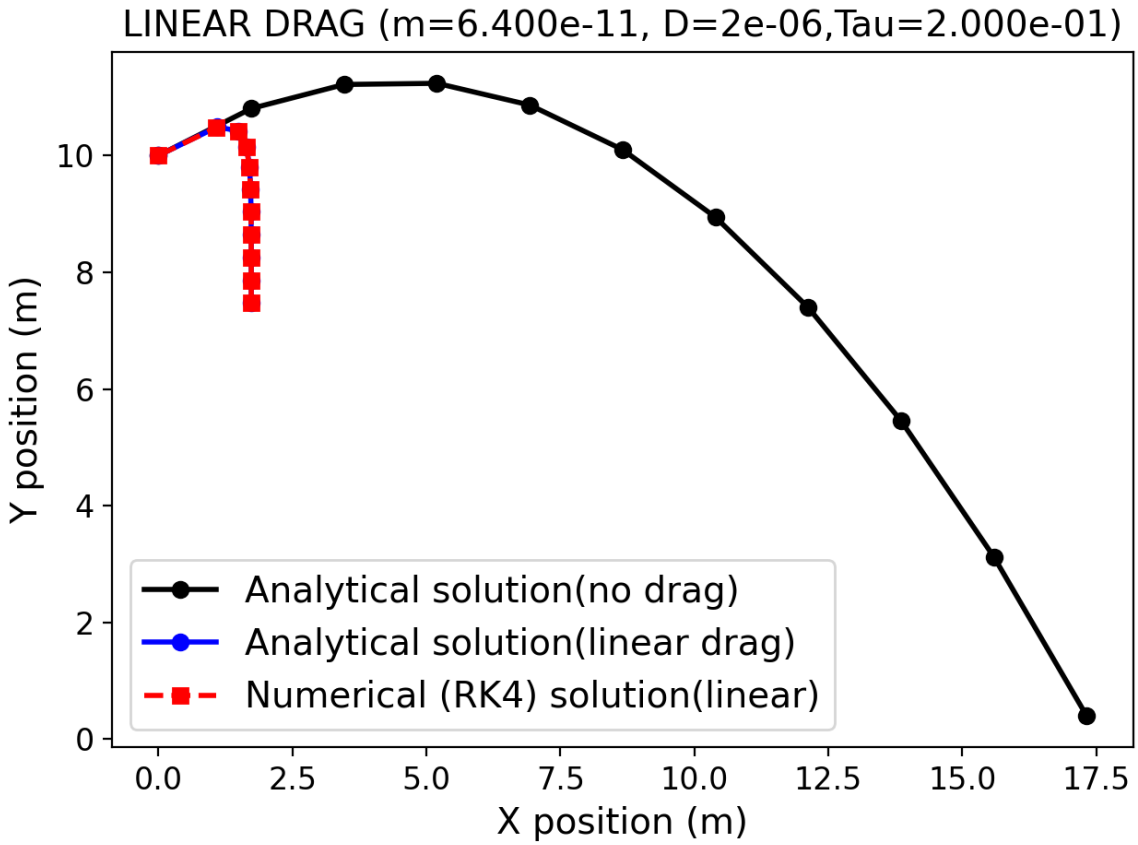

Figure 11: RK4 trajectories with linear drag for $\tau = \Delta t$
$\mathrm{m} = 6.400\mathrm{e}-11, \mathrm{D} = 2\mathrm{e}-06$, Tau $= 2.000\mathrm{e}-01$

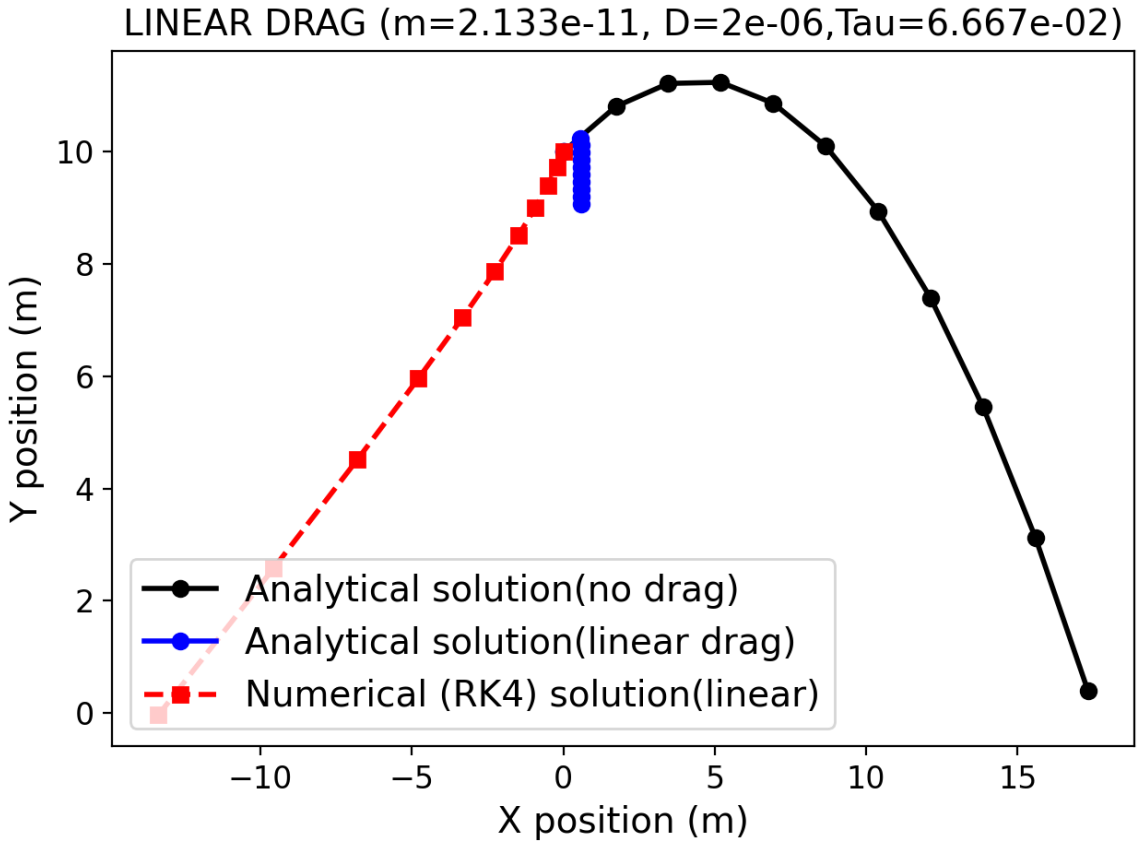

Figure 12: RK4 trajectories with linear drag for $\tau = \frac{1}{3}\Delta t$ and $dt = 0.2$.
$\mathrm{m} = 2.133\mathrm{e}-11, \mathrm{D} = 2\mathrm{e}-06, \mathrm{Tau} = 6.667\mathrm{e}-02$. The time step is smaller than the characteristic time so it fails to approach the motion

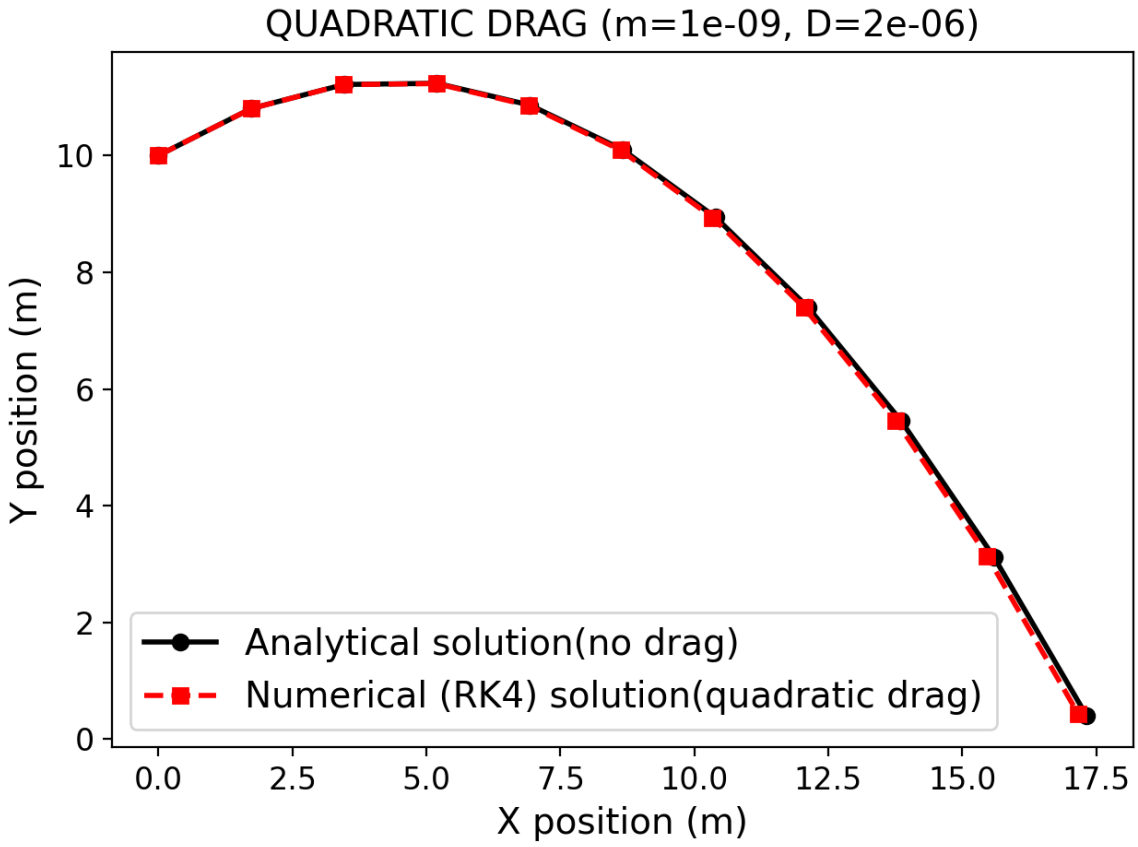

Figure 13: RK4 Trajectory for quadratic force and case of no air resistance. The mass and diameter are very small so linear drag dominant and quadratic drag not quit influences the motion
(Plot was generated by projectile3quad1.py)

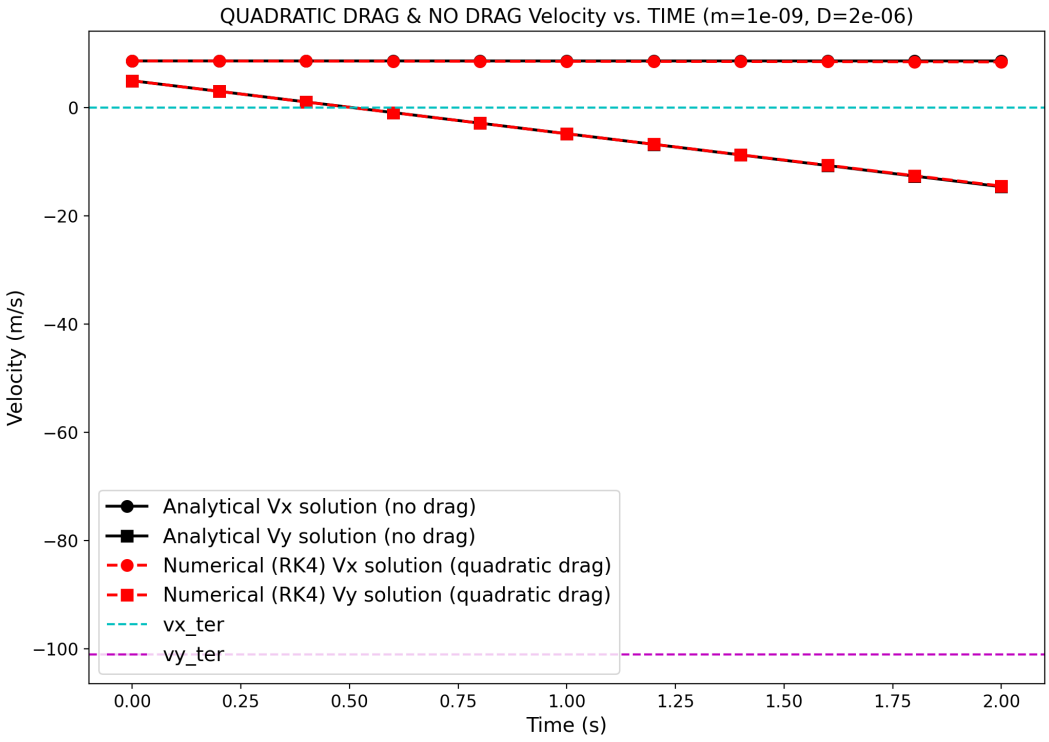

Figure 14: velocity evolution $vx(t)$ vs. t and $vy(t)$ vs. t for the quadratic drag case and the case of no air resistance
(Plot was generated by projectile3quad1.py)

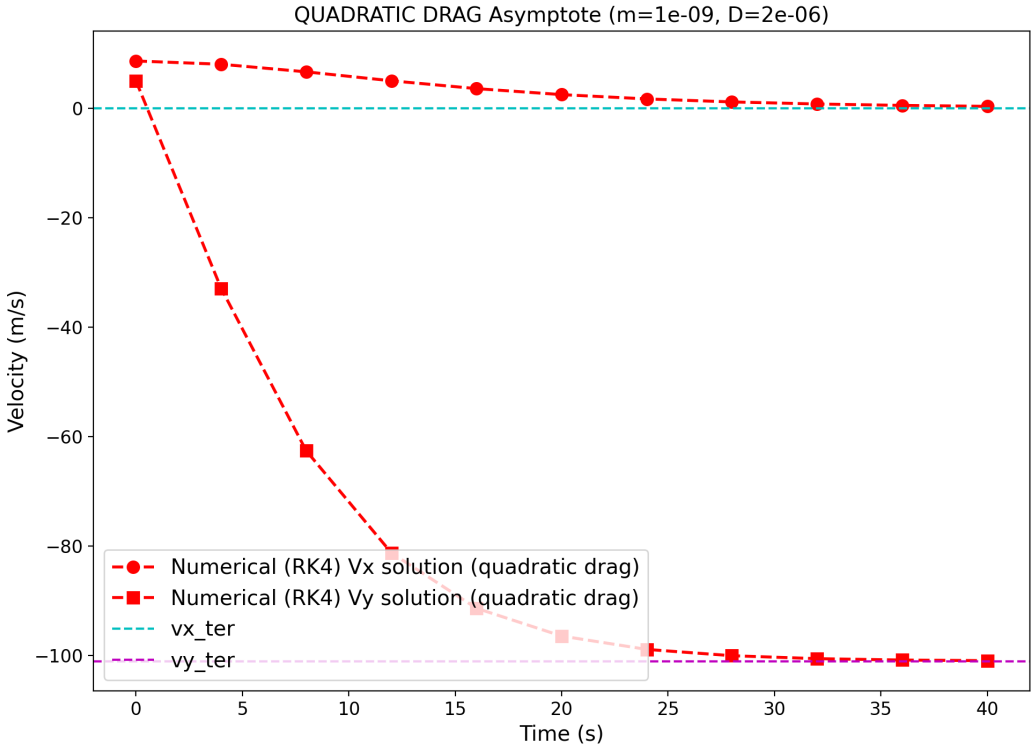

Figure 15: velocities asymptotically approach terminal velocity values when $t \to \infty$.
Plot was generated by projectile3quadAsymptote.py

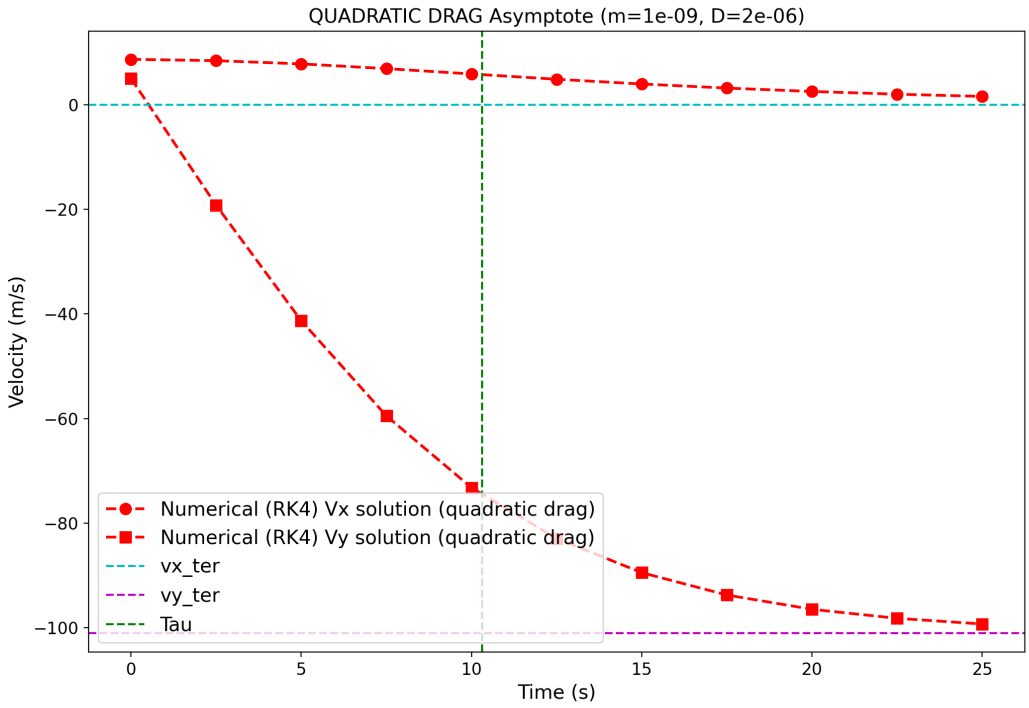

Figure 16: velocities asymptotically approach terminal velocity values when $t \to \infty$
$\tau_{quad} = \frac{m}{cv_{ter}} = 10.31s$
Plot was generated by projectile3quadAsymptote.py

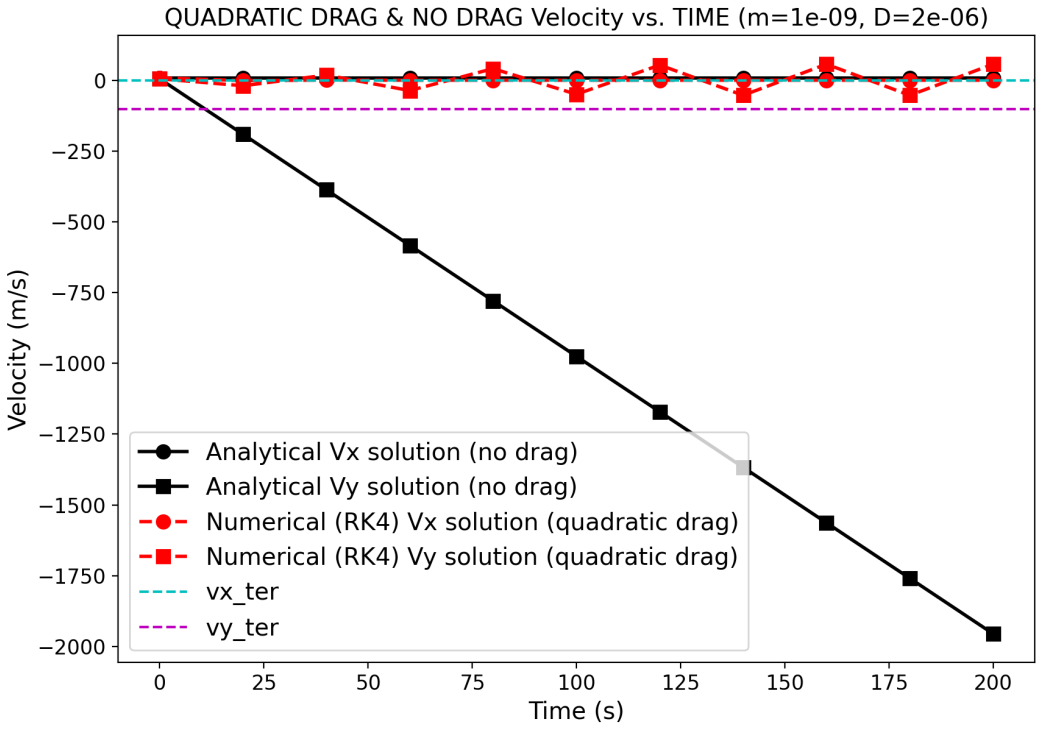

Figure 17: $\tau_{quad} = \frac{m}{cv_{ter}} = 10.31s < dt = 20s \quad \tau = 10.31$s. $Npts = 11 \quad tmax = 200s$ .So $dt = 20s$ and is greater than the 10.31s. The numerical method fails since dt is too big and model does not converge.
Plot was generated by projectile3quadAsymptote.py